Marrow Tech

Function reference: how every part of the code works

Marrow Tech is an online store for everyday tech, built with Flask and SQLite. This document goes through every function in the project, file by file: what it does, which URL it serves, and which rules it enforces.

File	Role	Functions
`app.py`	Starts the app, registers routes, the `create-admin` command	1 (+ startup code)
`store.py`	Database, schema upgrades, demo data, access checks	6
`routes/auth.py`	Register, sign in, sign out	4
`routes/front.py`	Storefront, customer pages, JSON API	15
`routes/admin.py`	Admin desk: products, categories, orders, users	12
`static/app.js`	Bag, checkout, invoice pop-up, admin tabs	8 + event handlers

Generated 03 November 2026. Line numbers point to each function's `def` line in the current code.

1. app.py: starting the app

When the module loads, before any function runs, it:

- Reads `APP_ENV` and stops with an error if production is set without a `SECRET_KEY`.
- Creates the Flask app and exposes it as `application`, the WSGI entry point that Gunicorn runs.
- Sets cookie security: `HttpOnly`, `SameSite=Lax`, and `Secure` in production.
- Registers the three blueprints (`front`, `auth`, `admin`).
- Adds **compatibility routes** (`compatibility_rules`, line 33): the same views are registered again under the old endpoint names (`storefront`, `login`, `admin`…), so old `url_for(...)` calls in templates keep working.
- Registers the `currency` template filter and the `current_user_context` context processor.
- Calls `init_db()` so the database is ready before the first request.

create_admin(name, email, password)

Line 69 · command: `flask --app app create-admin`

A command-line command that asks for a name, an email and a password (typed twice).

- The name and email are trimmed, and the email is lowercased.
- Rejects empty fields and passwords shorter than **12** characters.
- Hashes the password and inserts a user with `role = 'admin'`.
- If the email is already taken, the database error (`IntegrityError`) becomes a clear message: "An account with that email already exists."

if __name__ == "__main__"

Line 92

Starts the development server on `HOST`/`PORT` (default `127.0.0.1:8555`). Debug mode is on only when `FLASK_DEBUG` is true and the app is not in production.

2. store.py: database and shared helpers

Constants: `DATABASE` (path from `DATABASE_PATH`, or `shop.db`), `UPLOAD_DIR` (`static/uploads`), `PAGE_SIZE = 20`, the demo account credentials, and `SEED_PRODUCTS`. The sample products are 6 product types × 17 editions, cut to **100 products**; each edition costs $0–12 more than the base price.

get_db()

Line 33

Creates the database's folder if needed and opens a SQLite connection with `row_factory = sqlite3.Row`, so columns can be read by name (`row["price"]`). Every route opens and closes its own connection with this function.

init_db()

Line 40 · runs every time the app starts

Creates and upgrades the database:

- **Safety checks for production:** `DATABASE_PATH` must be set and must be an absolute path. `SEED_DEMO_DATA` must be true or false, and it cannot be on in production.
- **Tables:** creates `products`, `orders`, `users`, `favorites`, `reviews` and `categories` if they don't exist yet.
- **Upgrading older databases:** `PRAGMA table_info` checks which columns exist and adds missing ones: `orders.payment_method`; `products.image1–3`; `users.role`, `avatar`, `phone`, `address`, `card_name`, `card_last4`, `card_expiry`.
- Creates the uploads folder.

- **Demo data:** if the catalog is empty and demo data is allowed, inserts the demo admin (only if no admin exists yet), the demo staff account and 100 products.
- **Filling categories:** copies every category name used by a product into the `categories` table.

admin_access_required()

Line 88

Protects the admin desk. Returns `None` when access is allowed, or a response that blocks the request:

- Not signed in: the sign-in page with an error.
- Role not `admin` or `user` (staff): `access_denied.html`, status 403.

owner_admin_required()

Line 97

Runs the check above, then also requires `role == "admin"`. Only account creation uses it.

currency(value)

Line 106

Template filter: `1234.5` becomes `$1,234.50`.

current_user_context()

Line 110 · context processor, runs for every page

If someone is signed in, reloads their row from the database so changes such as a new photo or name show up immediately. Also loads their **5 most recent orders** as `notifications`, which feed the badge on the envelope icon in the header. Every template gets `current_user` and `notifications`.

3. routes/auth.py: accounts

safe_next_url(target)

Line 10

Prevents **open redirects**. A `?next=` value is only accepted if it is a path on this site: it starts with `/`, doesn't start with `//`, has no backslash, and has no scheme or host. Anything else returns `None`.

register()

Line 19 · GET/POST `/register`

- GET shows the sign-up form.
- POST checks that the name and email are present, the password is at least 6 characters, and both passwords match.
- Inserts the user with role `customer` and a hashed password. A duplicate email gets a friendly error.
- **Signs the new user in straight away** by storing `{id, name, email, role}` in the session, then redirects to `next` or to `/profile`.

login()

Line 40 · GET/POST `/login`

Looks up the user by lowercased email and checks the password hash with `check_password_hash`. The same error message is shown for a wrong email and a wrong password, so attackers can't tell which emails have accounts. On success it stores the user in the session and redirects to `next` or `/profile`.

logout()

Line 54 · GET `/logout`

Removes `user` from the session and goes back to the storefront.

4. routes/front.py: storefront, customer pages and JSON API

remove_old_avatar(avatar_url, connection, user_id)

Line 14

Deletes a profile photo after it has been replaced. It only deletes the file when all of these are true:

- It is a local path under `/static/uploads` with no scheme or host.
- The filename starts with `user-<id>-profile`, so it belongs to this user.
- No user avatar or product image still points to it.
- The real path on disk is inside the uploads folder, which blocks path tricks like `../`.

render_profile(error=None, status=200)

Line 40

Loads the account, the orders (matched by email) and the favorite products, and renders `profile.html`. `profile()` uses it for the normal page; `update_profile()` uses it to show the page again with a red error message and status 400 when the input is invalid.

storefront()

Line 49 · GET `/`

The home page. Reads `?q=` and `?page=`.

- With a search term, matches product name, description or category using `LIKE`.
- Shows 20 products per page, newest first, and loads the category list for the navigation.
- Sends `products`, `categories`, `page`, `total_pages` and `query` to `index.html`.

about_page()

Line 66 · GET `/about`

Shows the static about page.

category(category_name)

Line 70 · GET `/category/<name>`

Same as the storefront, but only shows products in that category, also 20 per page.

product_detail(product_id)

Line 81 · GET `/product/<id>`

Loads:

- The product, and its reviews joined with the reviewer's name.
- Up to 4 **recommendations** from the same category.
- Whether the current user has favorited it.
- Goes back to the storefront if the product doesn't exist, and calculates the **average rating** rounded to 1 decimal place.

checkout()

Line 94 · GET `/checkout`

- If nobody is signed in, redirects to sign-in with `next` set to this checkout URL, including any query string.
- Loads the user's saved address and saved card (last 4 digits only) to fill in the form.
- With `?buy_now=<id>` loads that single product for the **Buy now** flow. A missing or invalid ID goes back to the storefront.

profile()

Line 128 · GET `/profile`

Requires sign-in. Shows the account details, all orders (matched **by email**) and favorite products, through `render_profile()`.

notifications_page()

Line 134 · GET `/notifications`

Requires sign-in. Lists all of the user's orders with their status.

order_invoice(order_id)

Line 143 · GET `/api/orders/<id>` · JSON

Gives the data for the invoice pop-up.

- Not signed in: 401.
- Customers can only see their **own** orders; staff and admins can see any order. Otherwise it returns 404, so the endpoint doesn't reveal which orders exist.
- The `items` column is stored as a Python-style string, so it is read with `ast.literal_eval`, with a JSON fallback.
- Every item is checked (positive whole-number quantity, numeric price); broken items are skipped.

product_images()

Line 186 · GET `/api/products/images?ids=1,2,3` · JSON

Takes a comma-separated list of IDs, removes duplicates and allows 1–100 positive whole numbers (otherwise 400). Returns `{id: image1}` for products that have an image. The bag uses this to load missing pictures.

update_profile()

Line 204 · POST `/profile/update`

Requires sign-in. Updates name, email, phone, address, cardholder name, card expiry, and the avatar URL or an uploaded photo.

- **Checks first:** name and a valid email (with `@`) are required, and the email must not belong to another account. Problems are shown as a red message on the profile page (status 400) instead of an error page.
- **Card:** non-digits are removed, the length must be 13–19 digits, and **only the last 4 digits are stored**.
- **Photo:** only `.png/.jpg/.jpeg/.webp`. Each upload gets a unique name, `user-<id>-profile-<random id>.<ext>`, so browsers show the new photo straight away. The old photo is then deleted with `remove_old_avatar()`.
- Fields left empty keep their old values.
- If saving still fails because the email was taken at the same moment, the change is undone and the new photo file is deleted.
- The new name and email are written back into the session, so the header updates immediately.

toggle_favorite(product_id)

Line 259 · POST `/product/<id>/favorite`

Requires sign-in. Deletes the favorite if it exists or adds it if it doesn't, then goes back to the page the user came from.

add_review(product_id)

Line 270 · POST `/product/<id>/review`

Requires sign-in. A rating that isn't a whole number gets a 400 response, and a product that doesn't exist sends the user back to the storefront. Otherwise it saves the review with the rating limited to **1–5** and the comment trimmed.

create_order()

Line 288 · POST `/api/orders` and `/api/purchases` · JSON

The most important function. It records an order:

- **1.** Not signed in: 401 with a `login_url` that returns the user to the page they came from (checked to be a path on this site).
- **2.** Checks the request body: a JSON object with a `customer` object and an `items` list of 1–100 entries; `payment_method` must be `saved_card` or `khqr`; name, email (containing `@`) and address are required.
- **3.** Checks quantities: every item ID must be a positive whole number (not a boolean) and each quantity 1–99. Repeated IDs are added together, and the combined quantity still can't be more than 99.
- **4.** If the payment method is `saved_card`, the user must actually have a saved card.
- **5. Prices are looked up again in the database.** Prices sent by the browser are ignored, so nobody can change what they pay. If any product no longer exists, the order is rejected.
- **6.** Calculates the total and inserts the order with status `New`.
- **7.** Returns `purchase_id`, `total` and a message saying "Demo only; no payment was charged."

5. routes/admin.py: admin desk (/admin prefix)

product_image_inputs(form, files)

Line 13

Checks the 3 image slots of the product form. For each slot:

- Uploading a file **and** giving a link at the same time is an error.
- **A link** must be `http`/`https`, with a host, no username or password in it, no spaces, and a valid port.
- **A file** must be PNG, JPEG or WebP.
- Returns `{slot: ("url", link)}` or `{slot: ("upload", file, extension)}`, and raises `ValueError` with a readable message for bad input.

remove_unreferenced_product_image(image_url, connection, product_id, slot)

Line 45

Cleans up an old image after it has been replaced. It only deletes a file when all of these are true:

- It is a local path under `/static/uploads` with no scheme or host.
- The filename matches `product-<id>-<slot>`….
- No product or user avatar still points to it.
- The real path on disk is inside the uploads folder, which blocks path tricks like `../`.

dashboard()

Line 77 · GET `/admin`

Staff and admins only. Four tabs (`products`, `categories`, `orders`, `users`), chosen with `?view=`.

- The **users** tab only appears for admins; staff see a user count of 0.
- Each tab has its own page number (`products_page`, `orders_page`…), kept within the valid range.
- **Revenue** is the sum of all orders that are not `Cancelled`.

products()

Line 106 · GET `/admin/products`

The product manager: search (`?q=`) with page numbers that stay in range, plus a separate paged list of categories, each with its **product count** (`LEFT JOIN … GROUP BY`).

create_category()

Line 133 · POST `/admin/categories`

Adds a category. `INSERT OR IGNORE` means a duplicate name does nothing.

update_category(category_id)

Line 145 · POST `/admin/categories/<id>/edit`

Renames the category **and updates every product that uses the old name**, both in the same database transaction. If the new name already exists, the change is undone.

delete_category(category_id)

Line 163 · POST `/admin/categories/<id>/delete`

Only deletes the category if **no product uses it**.

add_product()

Line 175 · POST `/admin/products`

- Name and description are required; an invalid price becomes 0; the category must already exist.
- The product's **initials** come from the first letters of the first two words of its name ("Pulse Wireless" gives "PW"). The color defaults to `#d9c7bd`.
- Inserts the product first to get its ID, then saves uploaded images as `product-<id>-<slot>-<random id>.<ext>`.
- **If anything fails, everything is undone:** the database change is reversed and image files already saved are deleted.

edit_product(product_id)

Line 235 · POST `/admin/products/<id>/edit`

- Checks the category and the images; returns 404 if the product doesn't exist.
- Image slots left empty keep their current image.
- On failure it undoes the change in the same way as `add_product`.
- After saving, calls `remove_unreferenced_product_image` for every image slot that changed.

delete_product(product_id)

Line 304 · POST `/admin/products/<id>/delete`

Deletes the product **together with its reviews and favorites** in one transaction, then calls `remove_unreferenced_product_image` for each of its 3 images so the files are removed from disk. A product ID that doesn't exist just goes back to the product list.

update_order_status(order_id)

Line 325 · POST `/admin/orders/<id>/status`

Sets the status to one of `New`, `Packed`, `Shipped`, `Cancelled`. Any other value becomes `New`.

create_customer()

Line 337 · POST `/admin/customers`

Admins only (`owner_admin_required`). Creates a `customer` or `user` (staff) account. The password must be at least 6 characters and both entries must match. An `admin` account can't be created this way. A duplicate email is quietly ignored.

6. static/app.js: browser code

The **bag lives in** `localStorage` under the key `morrow-cart`, as a list of `{id, name, price, image, color, initials, quantity}`.

Function / handler	Line	What it does
`readCart()` / `writeCart(cart)`	2–3	Read and save the bag. `writeCart` also re-renders it.
`money(value)`	4	Formats a number as `$12.00`.
`renderCart()`	6	Updates every bag count and total on the page and draws the bag drawer items (remove button, image or initials, name link, price, quantity buttons). Builds the HTML element by element, so product names can't inject HTML (XSS-safe).
`hydrateCartImages()`	77	For bag items without an image, calls `/api/products/images` and saves the results into the bag.
`showToast(message)`	98	Shows a pop-up message for 2.8 seconds.
`toggleDrawer(open)`	103	Opens or closes the bag drawer and updates `aria-hidden` for screen readers.
`openInvoice(orderId)`	108	Opens the invoice pop-up, shows "Loading…", calls `/api/orders/<id>`, then fills in customer, address, date, status, payment label, item lines and total. Errors are shown inside the pop-up.
`closeInvoice()`	151	Hides the invoice pop-up.
Admin tabs (`activateAdminTab`)	163	Switches tab panels, updates `?view=` in the address bar without reloading, and supports keyboard navigation with the arrow keys, Home and End.
Global click handler	190	One handler for the whole page: opens and closes invoices, **adds to bag** (increases the quantity if the item is already there), opens and closes the drawer, changes quantities (items that reach 0 are removed), removes items, and opens or closes the quick-checkout pop-up (blocked when the bag is empty).
Escape key handler	229	Closes the invoice pop-up.
Quick-checkout submit handler	233	Sends the pop-up form to `/api/purchases`. A 401 redirects to sign-in. On success it clears the bag, closes the pop-up and shows the server's message.
Checkout page (`DOMContentLoaded`)	244	On `/checkout`, builds the order from the **Buy now** product or the bag, draws the summary and total, and fetches missing images. On submit it disables the button, posts the order, and either redirects to sign-in, shows an error, or shows the confirmation. It only clears the bag when the order came from the bag.
`renderCart()` at end of file	353	Draws the bag when each page loads.

7. How a purchase works from start to finish

1. **Add to bag** puts the item into `localStorage` and updates the count in the header.
2. **Checkout** sends a user who isn't signed in to `/login?next=/checkout…`, which brings them back afterwards.
3. Submitting the form sends `POST /api/purchases` with only `{id, quantity}` for each item.
4. `create_order` checks everything, reprices from the database and saves the order with status `New`.
5. The order appears in the customer's notifications and profile. Staff move it through Packed, Shipped (or Cancelled) in `/admin`, and the invoice is available through `/api/orders/<id>`.

8. Issues fixed after the first review

These problems were found while writing this reference and have since been fixed:

Issue	Fix
Changing your email to one another account uses crashed the profile page (error 500). Empty names and emails were accepted.	`update_profile()` checks the name and email and shows a clear error message on the profile page.
Changing your name or email didn't update the signed-in session, so the header kept showing the old values.	The session is rewritten with the new values after saving.
Deleting a product left its reviews, favorites and image files behind.	`delete_product()` deletes them together with the product.
A rating that wasn't a number crashed `add_review()`, and reviews for products that don't exist were saved.	Bad ratings get a 400 response; missing products are rejected.
A replaced profile photo kept the same filename, so browsers kept showing the cached old photo.	Each upload gets a unique filename and the old file is deleted.
`shop.db` and `__pycache__` files were committed to git.	Both are no longer tracked; `shop.db` is listed in `.gitignore`.

Still open: orders are linked to customers by email, so after an email change the earlier orders no longer show on the profile. Fixing that needs a `user_id` column on `orders`.